Status	Finished
Started	Thursday, 29 October 2026, 8:13 PM
Completed	Thursday, 29 October 2026, 8:21 PM
Duration	7 mins 59 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def modify_file_line():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # Read inputs
    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    # Read existing lines from the input file
    with open(filename, 'r') as f:
        lines = f.readlines()

    # Replace the specified line (converting 1-based index to 0-based index)
    if 1 <= line_number <= len(lines):
        # Ensure proper newline formatting on replacement
        lines[line_number - 1] = new_content.rstrip('\r\n') + '\n'

    # Write the modified content to output.txt
    with open('output.txt', 'w') as f:
        f.writelines(lines)

if __name__ == "__main__":
    modify_file_line()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`2`
		`Updated line two.`
			`Line one.`
			`Updated line two.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Updated line two.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`2`
		`Line B Updated.`
			`Line A.`
			`Line B Updated.`
			`Line C.`
				`Line A.`
				`Line B Updated.`
				`Line C.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
import sys

def read_specific_line(filename, line_number):
    try:
        with open(filename, 'r') as file:
            for current_line_number, line in enumerate(file, start=1):
                if current_line_number == line_number:
                    # Strip trailing newline character
                    return line.rstrip('\r\n')
    except FileNotFoundError:
        return ""
    return ""

def main():
    # Read stdin to handle competitive programming / online judge input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    content = read_specific_line(filename, line_number)
    if content:
        print(content)

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import sys
import string
from collections import Counter

def process_word_frequencies(text):
    # Split text into words, strip punctuation, and convert to lowercase
    raw_words = text.strip().split()
    cleaned_words = []

    for word in raw_words:
        # Strip trailing and leading punctuation (e.g. "world!" -> "world")
        cleaned = word.strip(string.punctuation).lower()
        if cleaned:
            cleaned_words.append(cleaned)

    # Count frequency of each word
    word_counts = Counter(cleaned_words)

    # Write word frequencies to output.txt in alphabetical order
    with open('output.txt', 'w') as file:
        for word in sorted(word_counts.keys()):
            file.write(f"{word}: {word_counts[word]}\n")

def main():
    # Read text input from standard input
    input_data = sys.stdin.read()
    if not input_data:
        return

    process_word_frequencies(input_data)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r')`
	`as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import sys
import string

def find_longest_word(filename):
    longest_word = ""

    try:
        with open(filename, 'r') as file:
            for line in file:
                # Split line into words
                words = line.strip().split()
                for word in words:
                    # Remove punctuation marks around/in the word
                    cleaned_word = word.strip(string.punctuation)
                    # Update if a longer word is found
                    if len(cleaned_word) > len(longest_word):
                        longest_word = cleaned_word
    except FileNotFoundError:
        return ""

    return longest_word

def main():
    # Read standard input for filename
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    longest = find_longest_word(filename)

    if longest:
        print(f"Longest word: {longest}")

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
import sys

def insert_line_in_file(filename, line_number, new_content):
    # Read all lines from the input file
    with open(filename, 'r') as file:
        lines = [line.rstrip('\r\n') for line in file]

    # Convert 1-based line number to 0-based index
    target_index = line_number - 1

    # Insert the new line at the specified position
    if target_index < len(lines):
        lines.insert(target_index, new_content)
    else:
        lines.append(new_content)

    # Write all lines back with explicit newline endings
    with open('output.txt', 'w') as file:
        for line in lines:
            file.write(line + '\n')

def main():
    # Read standard input to handle dynamic test runner inputs
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())
    new_content = input_data[2]

    insert_line_in_file(filename, line_number, new_content)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`3`
		`Inserted line.`
			`Line one.`
			`Line two.`
			`Inserted line.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Line two.`
				`Inserted line.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`4`
		`Inserted line D.`
			`Line A.`
			`Line B.`
			`Line C.`
			`Inserted line D.`
				`Line A.`
				`Line B.`
				`Line C.`
				`Inserted line D.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
import sys

def count_words_in_file(filename):
    total_words = 0
    try:
        with open(filename, 'r') as file:
            for line in file:
                # Split line by whitespace and accumulate total count
                words = line.strip().split()
                total_words += len(words)
    except FileNotFoundError:
        return 0
    return total_words

def main():
    # Read standard input for filename
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    word_count = count_words_in_file(filename)

    print(f"Total words: {word_count}")

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import sys
import string

def is_palindrome(word):
    # Palindrome check (case-insensitive)
    normalized = word.lower()
    return len(normalized) > 0 and normalized == normalized[::-1]

def find_and_write_palindromes(filename):
    palindromes = []
    seen = set()

    try:
        with open(filename, 'r') as file:
            for line in file:
                words = line.strip().split()
                for w in words:
                    # Strip leading and trailing punctuation
                    cleaned = w.strip(string.punctuation)
                    if cleaned and is_palindrome(cleaned):
                        # Lowercase version for duplicate checking to maintain unique palindromes
                        lowered = cleaned.lower()
                        if lowered not in seen:
                            seen.add(lowered)
                            palindromes.append(lowered)
    except FileNotFoundError:
        pass

    # Write found palindromes to output.txt
    with open('output.txt', 'w') as file:
        for pal in palindromes:
            file.write(f"{pal}\n")

def main():
    # Read standard input for filename
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    find_and_write_palindromes(filename)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`output1.txt`
		`This is the source file.`
		`It contains multiple lines of text.`
		`Here is another line.`

Answer: (penalty regime: 0 %)

```
import sys

def copy_file_contents(source_filename, dest_filename):
    try:
        # Read content from the source file
        with open(source_filename, 'r') as source_file:
            content = source_file.read()

        # Write content to the destination file
        with open(dest_filename, 'w') as dest_file:
            dest_file.write(content)

    except FileNotFoundError:
        pass

def main():
    # Read standard input for source and destination file names
    input_data = sys.stdin.read().splitlines()
    if len(input_data) < 2:
        return

    source_filename = input_data[0].strip()
    dest_filename = input_data[1].strip()

    copy_file_contents(source_filename, dest_filename)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`output1.txt`
			`This is the source file.`
			`It contains multiple lines of text.`
			`Here is another line.`
				`This is the source file.`
				`It contains multiple lines of text.`
				`Here is another line.`
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`output2.txt`
			`Hello, world!`
			`Python programming is amazing.`
			`Let's copy this text to another file.`
				`Hello, world!`
				`Python programming is amazing.`
				`Let's copy this text to another file.`
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input3.txt`
		`output3.txt`
			`Single line.`	`Single line.`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys

def delete_line_from_file(filename, line_number):
    # Read all lines from the input file
    with open(filename, 'r') as file:
        lines = [line.rstrip('\r\n') for line in file]

    # Convert 1-based line number to 0-based index
    target_index = line_number - 1

    # Remove the target line if index is within valid bounds
    if 0 <= target_index < len(lines):
        lines.pop(target_index)

    # Write remaining lines to output.txt with proper line endings
    with open('output.txt', 'w') as file:
        for line in lines:
            file.write(line + '\n')

def main():
    # Read standard input to handle dynamic test runner inputs
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    delete_line_from_file(filename, line_number)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		2
			Line one.
			Line three.
			Line four.
				Line one.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		3
			Line A.
			Line B.
				Line A.
				Line B.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	3
		Line one.
		Line two.
		eerht eniL.
		Line four.

Answer: (penalty regime: 0 %)

```
import sys
import string

def reverse_line_content(text_line):
    # Separate trailing punctuation from the end of the line
    punctuation = ""
    while text_line and text_line[-1] in string.punctuation:
        punctuation = text_line[-1] + punctuation
        text_line = text_line[:-1]

    # Reverse the main text content and re-attach trailing punctuation
    return text_line[::-1] + punctuation

def reverse_line_in_file(filename, line_number):
    # Read all lines from the input file and strip trailing line breaks
    with open(filename, 'r') as file:
        lines = [line.rstrip('\r\n') for line in file]

    # Convert 1-based line number to 0-based index
    target_index = line_number - 1

    # Reverse the words/characters while preserving trailing punctuation
    if 0 <= target_index < len(lines):
        lines[target_index] = reverse_line_content(lines[target_index])

    # Write all lines back to output.txt with explicit line endings
    with open('output.txt', 'w') as file:
        for line in lines:
            file.write(line + '\n')

def main():
    # Read standard input for filename and line number
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    filename = input_data[0].strip()
    line_number = int(input_data[1].strip())

    reverse_line_in_file(filename, line_number)

if __name__ == "__main__":
    main()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.